IMSET
INSTITUT MAGHRÉBIN DES SCIENCES ÉCONOMIQUES ET DE TECHNOLOGIE
HONORIS UNITED UNIVERSITIES

ACADÉMIE D'ART DE CARTHAGE
HONORIS UNITED UNIVERSITIES

Intitulé du cours: Programmation Java

Chapitre 3 :
Eléments Syntaxiques de Java

Filière: 2ème année – BTS Informatique de Gestion

Enseignante: Abid Arij

Année de formation 2024/2025

Plan

Objectifs du module
Introduction

–Syntaxe Java

Les types primitifs

Les tableaux

Les chaines de caractères

Les listes (ex ArrayList)

Les entrés sorties :

Sortie standard : System.out

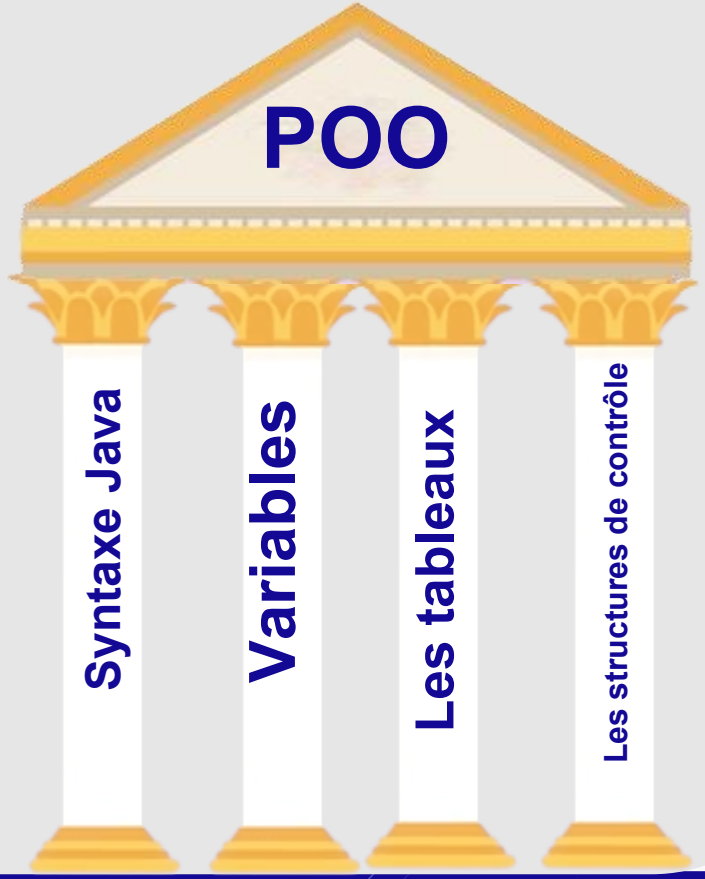
Entrée standard : (ex : la classe scanner)

Objectifs du module

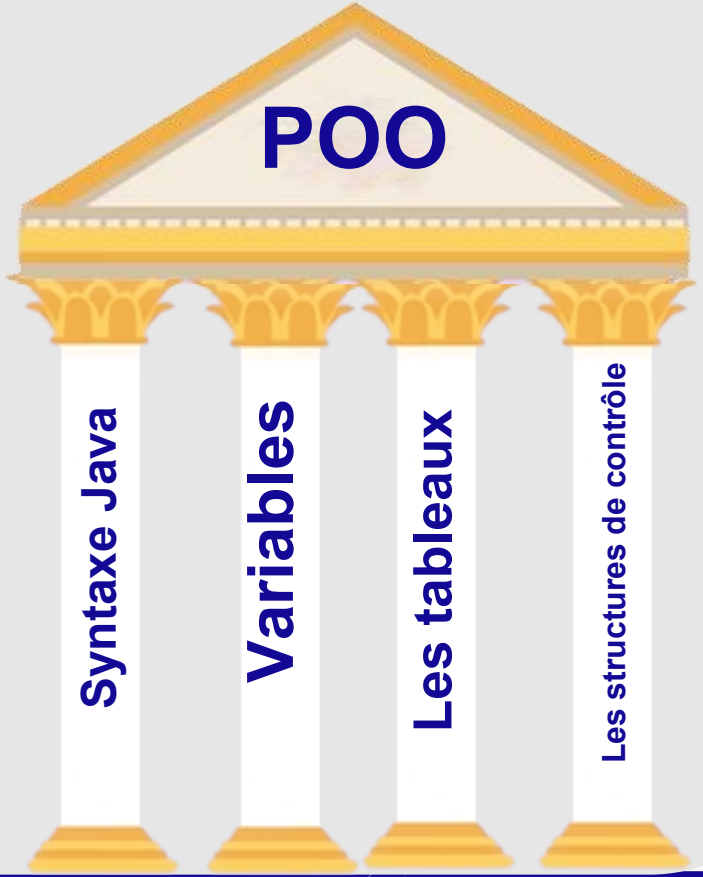
À la fin du module, l'apprenant doit être capable de :

- Comprendre la syntaxe de base du langage Java.
- Déclarer et utiliser des variables de différents types (primitifs et objets).
- Manipuler des tableaux et des chaînes de caractères en Java.
- Utiliser les structures de contrôle (conditions, boucles).
- Gérer les entrées et sorties standard avec la classe Scanner et System.out.
- Se familiariser avec les listes dynamiques comme ArrayList.

Introduction

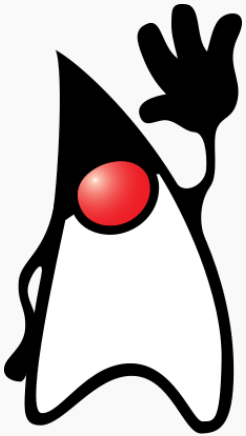


Syntaxe Java



Syntaxe Java

Java est un langage de programmation orienté objet, ce qui signifie que tout est organisé autour d'objets.
La syntaxe de base est assez similaire à celle d'autres langages comme C ou C++.



Structure de base d'un programme Java :

```
// HelloWorld.java

public class HelloWorld {
  public static void main(String[] args){
        System.out.println("Hello World!");
  }
}
```

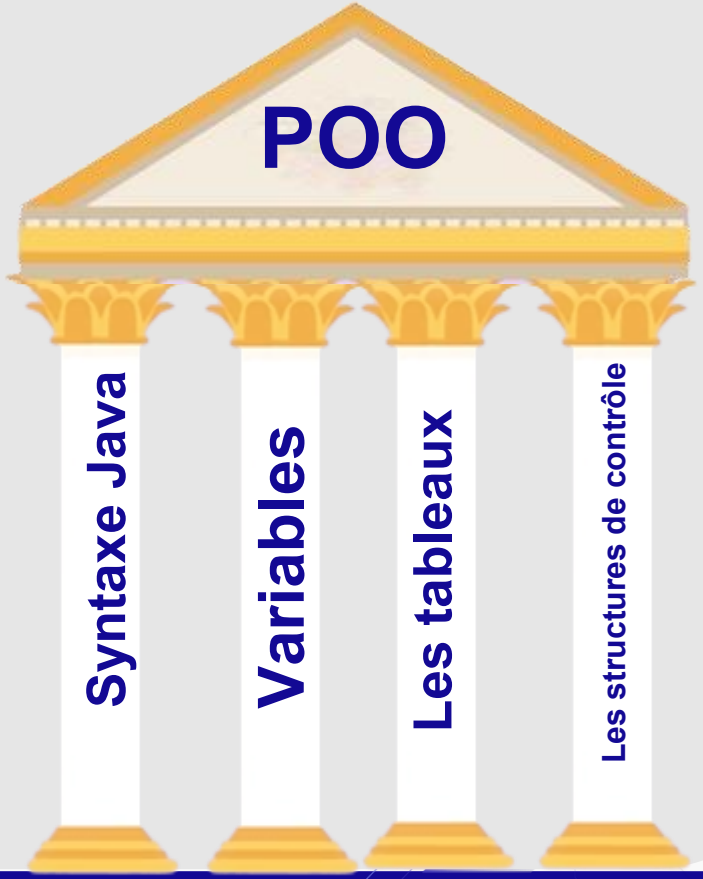
public class HelloWorld Déclare une classe appelée HelloWorld.

public static void main(String[] args): C'est la méthode principale d'un programme Java.

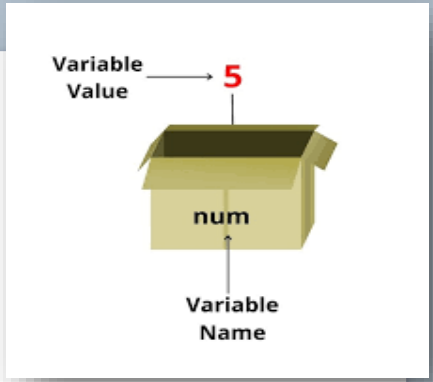
C'est à partir de cette méthode que l'exécution commence.

- System.out.println(): Cette fonction permet d'afficher un message dans la sortie standard (généralement la console).

Manipulation de variables



Notion de variable:



Dans les langages de programmation une **variable** sert à stocker la valeur d'une donnée

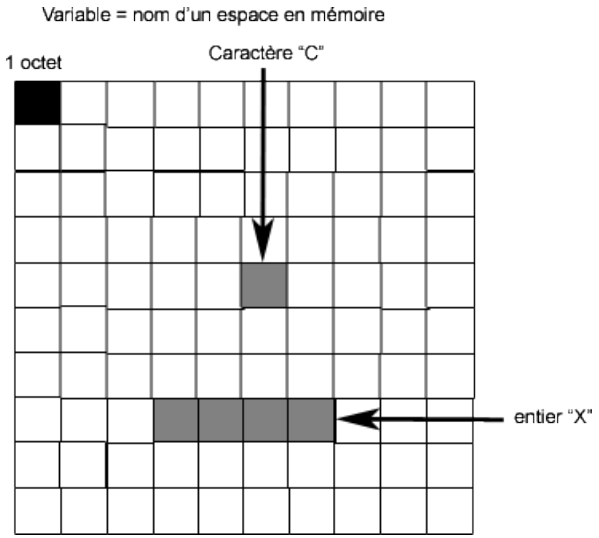
Une variable désigne en fait un emplacement mémoire dont le contenu peut changer au cours d'un programme (d'où le nom variable)

Règle : Les variables doivent être **déclarées** avant d'être utilisées, elle doivent être **caractérisées par :**

- **un nom (Identificateur)**
- **un type (entier, réel, caractère, chaîne de caractères, …)**
- **Une valeur**

variable

- Un nom
- une valeur
- un type



(la valeur d'une variable peut changer au cours de l'exécution).

Déclaration :

type nomDeLaVariable = valeur;

Exemple: int age = 25;

Une expression, pour un type, est soit une constante, soit une variable,
soit constituée à l'aide de constantes, de variables, de parenthèses et des opérateurs.

Syntaxe générale de Déclaration d’une variable :

🛠️ **3. Déclaration d’une variable**

<u>Syntaxe générale :</u>

type nomDeLaVariable = valeur;

Déclaration des variables

- ✓ Une variable possède un nom, un type et une valeur.
- ✓ La déclaration d'une variable doit donc contenir deux choses : un nom et le type de données qu'elle peut contenir.
- ✓ Le type d'une variable peut être:
  - soit un type primitif dit déclarée sous la forme type_primitif variable;
  - soit une classe déclarée sous la forme classe variable;

Exemple de déclaration:
boolean test;
String chaine;
int jour, mois, annee ;

Déclaration des variables

Quel nom choisir pour notre variable?

Les noms des variables sont case-sensitive

Les espaces ne sont pas permis

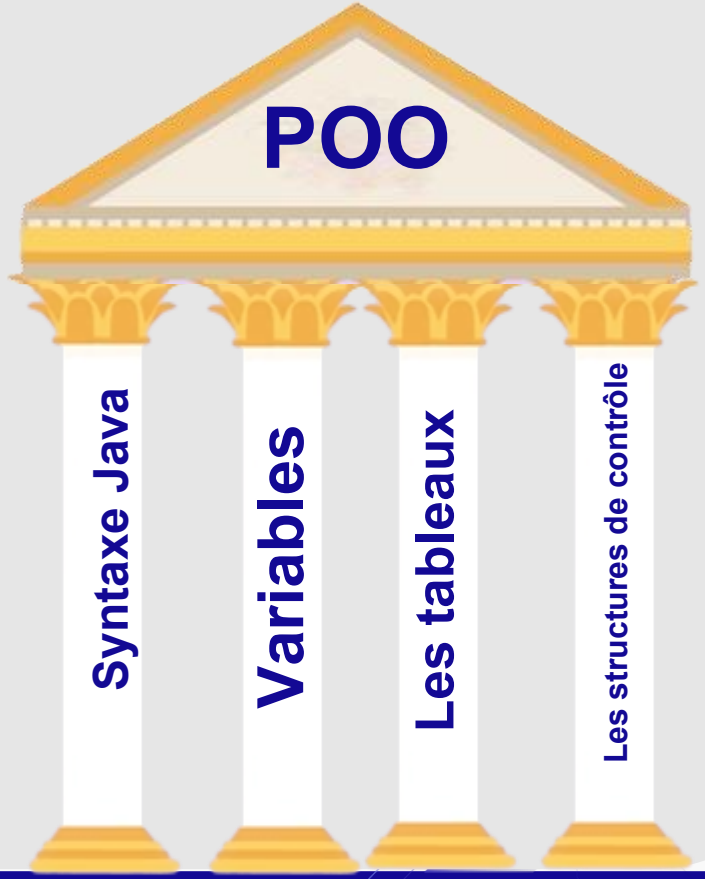
Le nom de la variable doit commencer par une lettre miniscule.

Syntaxe:

type nom_variable =valeur;

int id = 0;

Les types en java



Les types en java

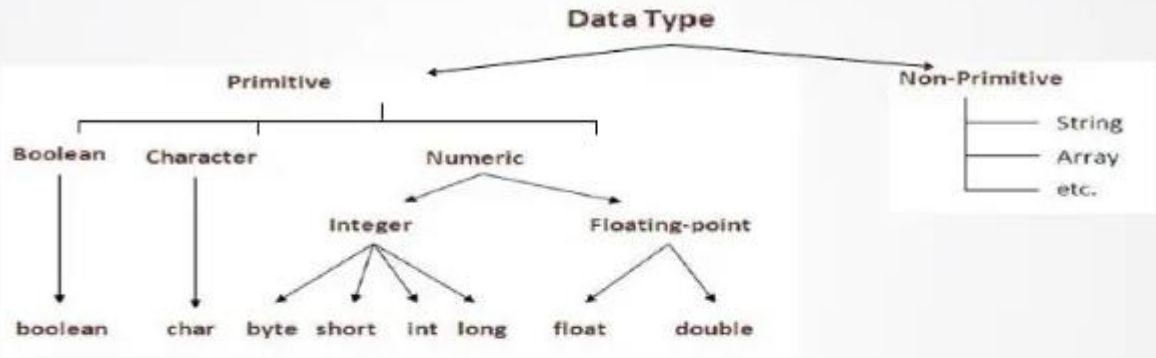
🌟 **Les Types de Variables en Java**
En Java, une **variable** est un espace mémoire qui permet de stocker une valeur.
Chaque variable a un **type** qui détermine la nature des données qu'elle peut contenir.
Il existe deux **grandes catégories** de types de variables en Java :

Java est un langage fortement typé.

Deux Types de données :

- ✓ Types primitifs
- ✓ Types de référence vers des instances d'une classe.

Les types en java



Il existe deux sortes de variables :

- Une variable primitive contient des bits qui représentent sa valeur.
  Exemple : int, foat , boolean.
- Une variable référence contient des bits qui indiquent comment accéder à l'objet.

Déclaration des variables

- **Constante**

  - On déclare une constante avec le mot final

    Exemple:

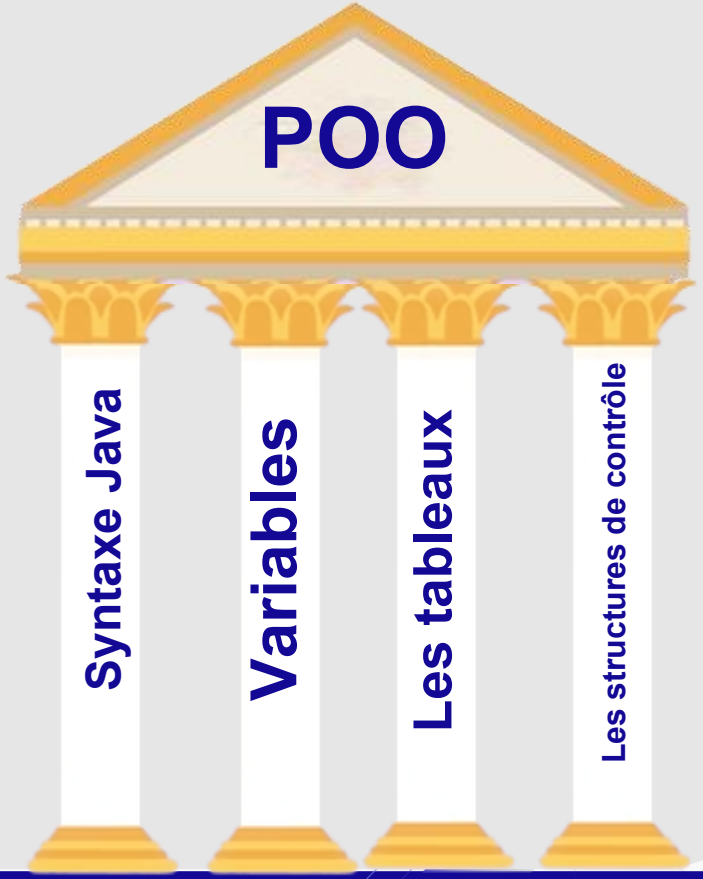
    ```
    final int CAPACITE=2000;
    ```

  - Le nom de la constante doit être en majuscule.
  - Si le nom est composé de plusieurs mots, on utilise _ pour la séparation des mots

  Exemple:

  ```
  final int TAILLE;
  final int MAX_STOCK;
  ```

Les types primitifs



2. Les types primitifs

Type	Taille	Valeurs possibles	Exemple
byte	1 octet (8 bits)	-128 à 127	byte age = 25;
short	2 octets	-32 768 à 32 767	short temperature = 15000;
int	4 octets	-2 milliards à 2 milliards	int population = 1000000;
long	8 octets	Très grandes valeurs entières	long distance = 12345678900L;
float	4 octets	Nombres décimaux (précis à ~7 chiffres)	float price = 12.5f;
double	8 octets	Nombres décimaux (plus précis)	double pi = 3.1415926535;
char	2 octets	Un seul caractère Unicode	char lettre = 'A';
boolean	1 bit (en théorie)	true ou false	boolean isOnline = true;

◆ Remarques :

- Les types float se terminent par f (ex. 12.5f)
- Les types long se terminent par L (ex. 100000L)

2. Types Référencés (ou Objets)

Les types **non primitifs** sont des **objets** ou des **références vers des objets**.
Exemples :

- String → pour les textes
- Array → pour les tableaux
- Classes personnalisées → créées par le développeur

Type	Description	Exemple
String	Chaîne de caractères	String nom = "Alice";
Array	Tableau (collection d'éléments)	int[] notes = {10, 15, 18};
Classes	Tout objet créé à partir d'une classe	Personne p = new Personne();
Interfaces	Contrats pour des classes	Runnable r = new MonRunnable();

Déclaration, Initialisation et Assignation des variables

Exemple:

`int t;`	Déclaration d'un entier t
`int u = 3;`	Déclaration et initialisation d'un entier
`int a = 1.2;`	Erreur de compilation
`t=7;`	Initialisation de t à la valeur 7
`u=t;`	Assignation (affectation) de la valeur de t à u
`m=9;`	Erreur déclaration préalable nécessaire
`char c;`	Déclaration
`c='a';`	Initialisation

Les types chaine de caractère

les chaînes de caractères

- ✓ les chaînes de caractères sont des objets. Ce sont des instances de la classe *String*.
- ✓ Déclaration de deux String :

```
String s1, s2;
```

- ✓ Initialisation :

```
s1 = "Hello";
s2 = "le monde";
```

- ✓ Déclaration et initialisation :

```
String s3 = "Hello";
```

- ✓ Concaténation :

```
String s4 = s1 + " " + s2;
```

Le type caractère char

```
char sexe='M';
```

Le type boolean

```
boolean Inscrit=true;
```

Le type chaine de caractère String

```
String message = ''Hello Word'';
```

En Java, un **opérateur** est un symbole qui effectue une opération sur une ou plusieurs variables.

Exemples : +, -, *, ==, &&, etc.

- On peut les classer en plusieurs **catégories** :

1. Opérateurs arithmétiques

- Ils servent à faire des calculs mathématiques.

Opérateur	Signification	Exemple	Résultat
+	Addition	5 + 3	8
-	Soustraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	6 / 3	2
%	Modulo (reste)	5 % 3	2

2. Opérateurs d'affectation

- Ils permettent de **donner une valeur** à une variable.

Opérateur	Signification	Exemple
=	Affectation simple	a = 5
+=	Ajoute puis affecte	a += 3; // a = a + 3
-=	Soustrait puis affecte	a -= 3; // a = a - 3
*=	Multiplie puis affecte	a *= 3; // a = a * 3
/=	Divise puis affecte	a /= 3; // a = a / 3
%=	Modulo puis affecte	a %= 3; // a = a % 3

3. Opérateurs de comparaison (relationnels)

- Ils comparent deux valeurs et donnent un **résultat booléen** (true ou false).

Opérateur	Signification	Exemple	Résultat
==	Égal à	5 == 5	true
!=	Différent de	5 != 3	true
>	Supérieur à	5 > 3	true
<	Inférieur à	5 < 3	false
>=	Supérieur ou égal à	5 >= 5	true
<=	Inférieur ou égal à	5 <= 3	false

4. Opérateurs logiques

- Ils permettent de **combiner plusieurs conditions**.

Opérateur	Signification	Exemple
&&	ET logique (vrai si les deux sont vrais)	(5 > 2) && (8 > 6) => true
!	NON logique (inverse)	!(5 > 2) => false

```
int age = 20;
boolean isAdult = (age >= 18) && (age <= 65);
System.out.println(isAdult); // true
```

5. Opérateurs unaires

- Ils agissent sur **une seule** variable.

Opérateur	Signification	Exemple
+	Plus (positif)	+a
-	Moins (négatif)	-a
++	Incrémentation (ajoute 1)	a++ ou ++a
--	Décrémentation (enlève 1)	a-- ou --a

Opérateurs

opérateur ***d'affectation***

=	affectation	
-=	soustraction et affectation	
+=	addition et affectation	

Opérateurs arithmétiques à deux opérandes

-	soustraction	
*	multiplication	
/	division	
%	modulo (reste de la division)	

Opérateurs à un opérande

-	opposé	-x	
++	pré-incrémentation		++x
++	post-incrémentation		x++
--	pré-décrémentation		--x
--	post-décrémentation		x--

Opérateurs relationnels

==	équivalent	x == 0	
<	plus petit que	x < 2	
>	plus grand que	x > 2	
<=	plus petit ou égal		x <= 3
>=	plus grand ou égal		x >= 3
!=	non équivalent	a != b	

Opérateurs logiques

&&	et	a && b
||	ou	a || b
!	non	!a

opérateurs ***d'arithmétique*** binaire

&	et	a & b	
|	ou	a | b	
^	ou exclusif		a ^ b
~	non	~a	
<<	décalage de bits à gauche		a << 2
>>	décalage de bits à droite		b >> 2

opérateur à trois opérandes

? :	condition ? vrai : faux	y < 5 ? 4 * y : 2 * y
(si…alors…sinon…)		

Les chaines de caractères

- Pour représenter une chaine de caractère, on utilise la classe `String` du package `java.lang`
- Les chaines de caractères n'ont pas de taille fixe.
- Déclaration d'une chaine :
  - `String s;` ou `String s1='LM';`
- La concaténation entre chaines avec +
- Il existe des méthodes associées à la classe `String`:
  - `s.length()` : retourne longueur de la chaine s
  - `s.equals(ch)` : compare la chaine s avec la chaine ch
  - `s.charAt(i)` : retourne le caractère de la position i de la chaine s
  - `s.indexOf(c)` : retourne la première position du caractère c dans s
  - `s.substring(d)` : retourne la sous-chaîne depuis d jusqu'à la fin
  - `s.substring(d,f)`: retourne la sous-chaîne depuis d jusqu'au caractère d'indice f non inclus.

Les chaines de caractéres

Méthode	Description	Exemple
length()	Renvoie la longueur de la chaîne	"Bonjour".length() → 7
charAt(int i)	Renvoie le caractère à l'indice i	"Java".charAt(0) → 'J'
toUpperCase()	Convertit en majuscules	"java".toUpperCase() → "JAVA"
toLowerCase()	Convertit en minuscules	"JAVA".toLowerCase() → "java"
substring(a, b)	Extrait une sous-chaîne	"Bonjour".substring(0, 3) → "Bon"
contains("mot")	Vérifie si la chaîne contient "mot"	"Java est cool".contains("cool") → true
equals("autre")	Compare deux chaînes (respecte la casse)	"Java".equals("java") → false
equalsIgnoreCase()	Compare sans tenir compte de la casse	"Java".equalsIgnoreCase("java") → true

Les chaines de caractéres

Commence toujours avec String.

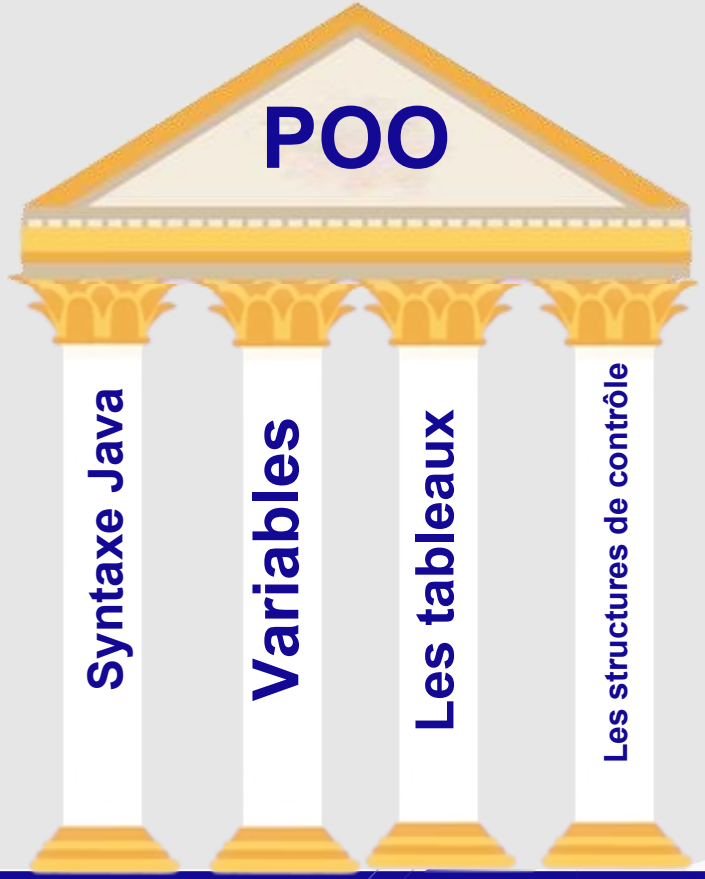
Utilise equals() et **pas ==** pour comparer deux chaînes !

```
String a = "Java";
String b = new String("Java");

System.out.println(a == b);       // false (compare les objets)
System.out.println(a.equals(b));   // true  (compare le contenu)
```

exemple

```
public class ExempleString {
    public static void main(String[] args) {
        String message = "Bonjour Java";

        System.out.println("Longueur : " + message.length());
        System.out.println("Première lettre : " + message.charAt(0));
        System.out.println("Majuscules : " + message.toUpperCase());
        System.out.println("Contient 'Java' ? " + message.contains("Java"));
    }
}
```

Les tableaux



Les tableaux

- Un tableau en Java est une structure de données qui permet de stocker plusieurs valeurs du même type.

Les tableaux

- Un tableau en Java doit être déclaré et initialisé avec une taille fixe avant d'être utilisé.

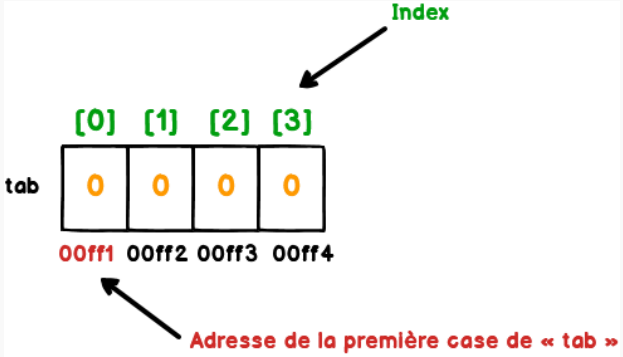
- Déclaration et allocation :

int[] tab; // Déclaration
tab = new int[10]; // Allocation

Ou en une seule étape :

int[] tab = new int[10];

Initialisation directe:
int t1[] = {10, 20, 30, 40};



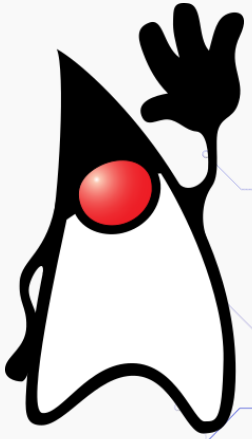


Tableau à plusieurs dimensions

- Lors d'une définition de tableau le nombre de crochets indique le nombre de dimensions du tableau.
  - `int t2[][] = new int[5][10] ;`
- t2 est un tableau à 2 dimensions composé de 5 lignes et 10 colonnes.
- Un tableau à plusieurs dimensions peut être déclaré et initialisé :
  - `int t3[][] = {{1, 2, 3, 4}, {5, 6, 7, 8}};`

- ✓ Une variable est déclarée comme un tableau dès lors que des crochets sont présents soit après son type, soit après son identificateur.

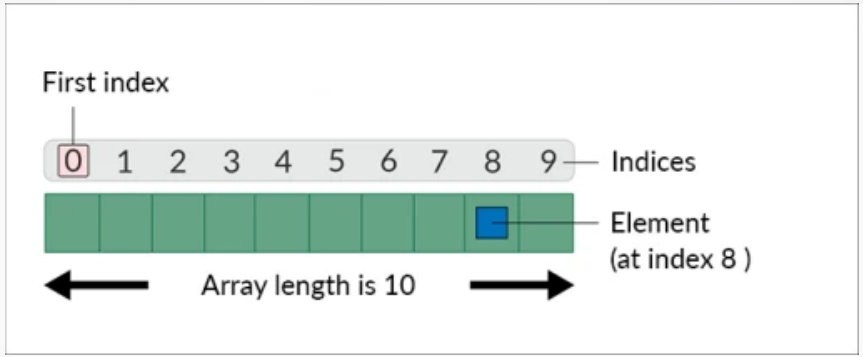
```
int [] myArray ; // ou bien
int myArray [];
```

- ✓ Un tableau a toujours une taille fixe qui doit être précisée avant l'affectation de valeurs :

  Par exemple : `int myArray [] = new int[20];` // indices de 0 à 19 (n-1)

- ✓ La taille d'un tableau est connue par l'attribut **length:** nom_tableau.length.

- ✓ On peut également créer des matrices ou des tableaux à deux dimensions en multipliant les crochets. Par exemple : `int[][] ma_matrice;`



Les listes

- Les listes sont des tableaux qui ont les caractéristiques suivantes :
  - Ils sont dynamiques (pas de taille fixe)
  - Peuvent contenir des objets de types différents
  - Possèdent des méthodes propres de manipulation
- Les listes sont définies dans le package `java.util`
- Il existe plusieurs types de liste, nous citons
  - ArrayList
  - Vector
  - …

ArrayList

- ArrayList est une classe d'objets qui :
  - n'ont pas de taille limite
  - Ils acceptent n'importe quel type de données, y compris null
- Création :

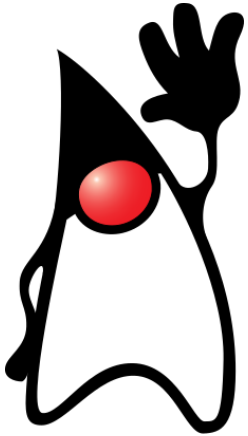
```
ArrayList al = new ArrayList();
```

- Ajouter un objet à la liste :

```
al.add(objet)
```

- Déterminer la taille de la liste:

```
al.size()
```



ArrayList

Autres méthodes :

- `get(i)` : retourne l'élément à d'indice `i` de la liste,
- `remove(i)` : efface l'élément d'indice `i`,
- `isEmpty()` : renvoie « vrai » si la liste est vide,
- `removeAll()` : efface tout le contenu de la liste,
- `contains(Object element)` : retourne `true` si l'élément passé en paramètre est dans l'ArrayList, `false` sinon. (utile pour la recherche).

Définition Entrée/Sortie

- Une entrée/sortie en Java consiste en un échange de données entre le programme et une autre source, par exemple l'écran, le clavier, un fichier,…
- Java met à disposition un ensemble d'objets permettant de réaliser cet échange.
- Java a décomposé les objets traitant les E/S en deux catégories :
  - les objets travaillant avec des flux d'entrée (`in`), pour la lecture de flux ;
  - les objets travaillant avec des flux de sortie (`out`), pour l'écriture de flux.

Définition Entrée/Sortie

- Une entrée/sortie en Java consiste en un échange de données entre le programme et une autre source, par exemple l'écran, le clavier, un fichier,...
- Java met à disposition un ensemble d'objets permettant de réaliser cet échange.
- Java a décomposé les objets traitant les E/S en deux catégories :
  - les objets travaillant avec des flux d'entrée (`in`), pour la lecture de flux ;
  - les objets travaillant avec des flux de sortie (`out`), pour l'écriture de flux.

Entrée standard (clavier)

- L'objet `System.in` correspond à l'entrée standard (clavier).
- En Java, pour indiquer qu'il faut lire ce qui est saisie au clavier on utilise un objet de la classe `Scanner` (du package `java.util`) qui est initialisé sur l'objet `System.in`
- Syntaxe :

```
Scanner sc = new Scanner(System.in);
```

- La classe `Scanner` met à la disposition un ensemble de méthodes pour lire des données à partir du clavier selon leurs types.

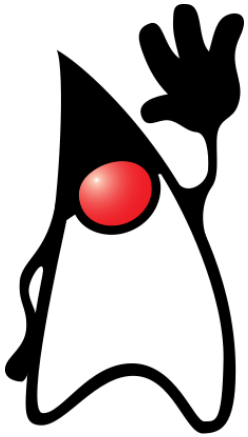
Entrée standard (clavier)

- Ces Méthodes de la classe Scanner retourne la valeur saisie:
  - `nextInt()` : pour la saisie de type `int`
  - `nextLong()` : pour la saisie de type `long`
  - `nextDouble()` : pour la saisie de type `double`
  - `nextLine()` : pour la saisie de type `String`
  - Pour saisir un caractère, on doit lire le premier caractère de la chaine saisie : `nextLine().charAt(0)`

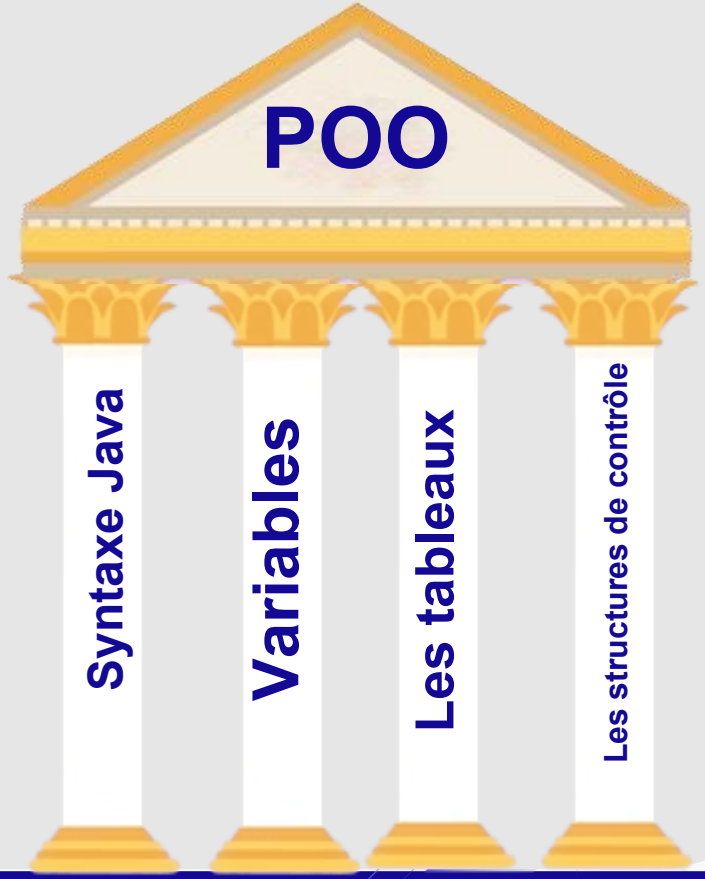
Entrée standard (clavier)

- Exemple :

```
Scanner sc = new Scanner(System.in);
System.out.println("donner un entier positif");
int n = sc.nextInt()
if(n < 0)
 System.out.println ("cet entier est
  negatif");
…
```



Les structures de contrôle



🌟 1. Structure conditionnelle : if / else if / else

- Elle permet d'exécuter des blocs de code **seulement si une condition est vraie**.

```
if (condition) {
    // Bloc de code si la condition est vraie
} else if (autreCondition) {
    // Bloc de code si la première condition est fausse et celle-ci est vraie
} else {
    // Bloc de code si aucune des conditions précédentes n'est vraie
}
```

Exemple:

```
int age = 18;

if (age < 18) {
   System.out.println("Tu es mineur.");
} else if (age == 18) {
   System.out.println("Tu viens d’avoir 18 ans !");
} else {
   System.out.println("Tu es majeur.");
}
```

🌟 2. Structure switch (choix multiples)

👉 **Utilité :**

- Elle permet d'exécuter un bloc de code en fonction de la **valeur d'une variable**.

```
switch (variable) {
    case valeur1:
        // instructions
        break;
    case valeur2:
        // instructions
        break;
    default:
        // instructions si aucune des valeurs ne correspond
}
```

Exemple:

```
int jour = 3;

switch (jour) {
    case 1:
        System.out.println("Lundi");
        break;
    case 2:
        System.out.println("Mardi");
        break;
    case 3:
        System.out.println("Mercredi");
        break;
    default:
        System.out.println("Jour inconnu");
}
```

🌟 A. Boucle for (connue d’avance)

👉 **Utilité :**

- Répète un bloc de code un **nombre connu de fois**.

Syntaxe :

```
for (initialisation; condition; incrémentation) {
    // bloc de code à répéter
}
```

Exemple:

```
for (int i = 0; i < 5; i++) {
    System.out.println("i vaut : " + i);
}
```

Ici, on affiche les chiffres de 0 à 4 (5 fois).

🌟 B. Boucle while (test avant)

👉 **Utilité :**

- Répète tant que la condition est **vraie**.

Syntaxe :

```
while (condition) {
    // instructions à répéter
}
```

Exemple:

```
int compteur = 0;

while (compteur < 3) {
   System.out.println("Compteur : " + compteur);
   compteur++;
}
```

Affiche compteur de 0 à 2.

🌟 C. Boucle do-while (test après)

👉 **Utilité :**

- Répète **au moins une fois**, puis continue tant que la condition est vraie.

Syntaxe :

```
do {
    // instructions
} while (condition);
```

Exemple:

```
int nombre = 1;

do {
   System.out.println("Nombre : " + nombre);
   nombre++;
} while (nombre <= 3);
```

Affiche 1, 2, 3.

Merci pour votre attention